The best keyboard layout for programming

A measured layer cost, and one symbol layer

Joshua Markle

University of Virginia · October 2026

Experiment report · 25 October 2026 · GECKO commit baad493 + this study · corne-36 · `programmer-typed` corpus (prose 45, Markdown 10, code as typed in an editor 45, Backspace 5.6 %) · profile `prog-cal`. Data and scripts: `bench/programming-layout-2026-10-25/`.

Abstract

A programmer types letters and about thirty symbols. On a small split keyboard most symbols live on a layer, and how much a layer press costs decides where they go. No study measures it. We measure the nearest thing, a held Shift, in 56,358 typists of the Aalto 136M data: it makes the next keystroke ×1.78 slower, as much as 3.7 same-finger bigrams. GECKO's `code` profile charged a press 27 times less. With that weight, an optimizer moves even `, . '` onto a layer; with the measured cost, that design is 30 % worse. We also model what a programmer actually types: an editor inserts block closers and completes identifiers, so typed code has half the } and _ of the source files. On corne-36 with letters on the base and the measured layer cost, the optimum keeps `' = , .` on the base and puts **every other symbol and every digit on one held layer**; digits 1–0 in a row on that layer cost 0.8 %. On the same Colemak-DH letters, an optimized layer beats Pascal Getreuer's hand-made symbol layer by only 0.5 %; a US-style layer (the shifted number row) is 7 % behind and Miryoku 14 %. Re-optimizing the letters gains 3.6 % more. Single-language optima take their language's symbols onto the base (`:` for Rust, _ for Python) and lose 7–17 % on mixed work. **The limits matter:** the layer cost comes from Shift, not from a thumb-held layer; all differences between good designs are under 0.3 % of typing time, too small to claim a speed difference; and the search is not exact (seeds agree within 0.25 % only at 160 M evaluations).

1. Question and setup

Question. A programmer types letters and about 30 symbols. On a small split keyboard most symbols must sit on a layer. Which arrangement of letters, symbols and digits on base and layers is best, and how does it compare with published symbol layers?

Board. corne-36: 3 × 5 keys per hand plus three thumb keys each, the board where symbol layers matter most. Thumbs hold Space, Backspace and Enter (a constraint), Shift, and two layer keys. Letters stay on the base layer (a constraint). Everything else (the four free base keys, Shift pairs, the two layers' 66 keys) is searched.

Text, as typed. Source code is not what a programmer types. An editor inserts the closing brace of a block and completes long identifiers. GECKO's code corpora count the files, so a new corpus option models editor typing (`corpus build --editor autoclose,complete=4`): a closer that starts a line is dropped (the editor put it there when its opener and Enter were typed), and an identifier longer than four characters is typed up to four and then accepted. Typed code is 26 % shorter than the files, almost all of it letters, and } falls by half. `programmer-typed` mixes English prose (45), Markdown (10) and this typed code (45; Rust, Python and JS/TS 25 each, shell 9, C 8, Lua 8), plus Backspace from GECKO's error model.

Cost. GECKO's evidence-fitted `default` weights, the `code` profile's symbol terms (awkward symbol bigrams, bracket pairs adjacent or mirrored), and the cost of a layer press, which the next section measures.

Search. BLS, 40 M evaluations, 6 seeds per cell, 4 threads. Each cell starts from the US-style design; a second grid starts from the best result.

Designs compared (`refs.py`; same thumbs, same Colemak-DH base with `' , . /` and their US Shift pairs, so only the layers differ):

- **Miryoku** (manna-harbour): symbols and a numpad on the left hand, held by a right thumb; the symbol layer holds the shifted symbols of the number keys.
- **Getreuer v2** (Pascal Getreuer's "Designing a symbol layer"): comparisons, arithmetic and brackets in groups, index fingers on = and (, inward rolls for != <= +=; a numpad on the left of a second layer.
- **US-style**: the shifted number row (`! @ # $ % ^ & * ( )`) on the layer's top row, the other US symbol keys below, digits in a row on the second layer: what most people build first.

Published designs duplicate keys (Miryoku's (, Getreuer's `, . '`); GECKO places each symbol once, so the duplicate is left empty. Sources and design rules: `research/sources/symbol-layers.md`.

2. What a layer press costs

GECKO counts layer presses (`layer_switch`: a hold counts once per run of layer keys, a one-shot once per key, a toggle twice per run), but its `code` profile weighted a press at 1. No study measures layer-key timing (`research/sources/symbol-layers.md`). The nearest thing that is measured at scale is Shift: a held modifier pressed by another finger before the key. The Aalto 136M data has the key-down times of 56,358 QWERTY touch typists (`shift.py`).

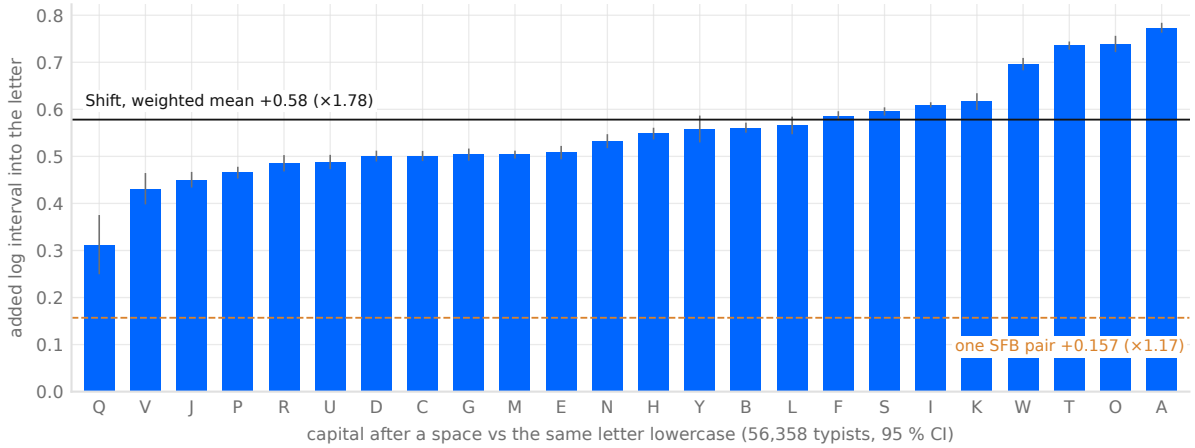


Figure 1: What a held Shift adds to the interval into a letter: a capital after a space against the same letter in lowercase after a space, per letter, with 95 % intervals. The weighted mean is +0.578 log interval (×1.78, 120,361 capitals). An SFB pair costs +0.157 on the same scale (orange).

- **Into a capital:** +0.578 log interval, ×1.78 the time of the same letter in lowercase. Left-hand letters +0.595, right-hand +0.559, so the cost hardly depends on which hand holds Shift.
- **Into a shifted symbol:** letter → ? against letter → . (same key): +0.590. A shifted symbol costs the same as a capital.
- **Out of it:** the letter after a capital is slower by +0.40, but names after a capital confound it, so it is left out.
- Shift goes down a median 173 ms after the previous key and 112 ms before the letter. It is still held past the next key in 12 % of cases.

Calibration. An SFB pair adds +0.157 on the same scale, and `default` charges 3.626 points per SFB %. One press is worth 0.578 / 0.157 = 3.68 SFBs, so `layer_switch` (per 100 bigrams, scale 2) gets weight **26.8 instead of 1**. That gives `prog-cal`; `prog-old` keeps the old weight. One score point is then ≈ 0.043 % of typing time per keystroke, which turns score gaps into time. The calibration assumes a layer key costs what Shift costs. A thumb-held layer may cost less (thumbs are idle) or more (layer keys are rarer and less practised); either way it is the only measured anchor.

Why it matters. Under the old weight, the optimum (`full-old`) empties four base keys and moves `, . '` to the layer, where frequent symbols are cheaper than any base key. Under the measured cost, that design is 30 % worse than the optimum: 13.5 layer presses per 100 bigrams against 9.9.

3. The optimum

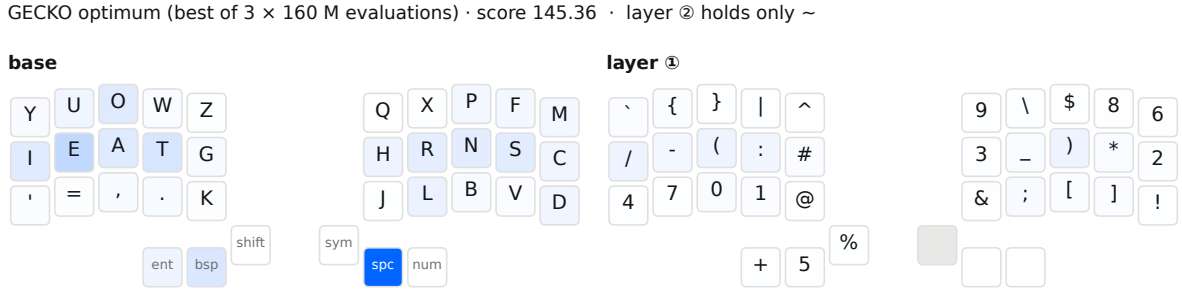


Figure 2: The optimum on corne-36: base layer (left) and its symbol layer (right), held by the right inner thumb. Keys are shaded by keystroke share in `programmer-typed` (layer keys on a 3× finer scale). Layer ② (right outer thumb) holds only ~.

```
base                        layer L1 (held: right inner thumb)
y u o w z   q x p f m       ` { } | ^   9 \ $ 8 6
i e a t g   h r n s c       / - ( : #   3 _ ) * 2
' = , . k   j l b v d       4 7 0 1 @   & ; [ ] !
 ent bsp sft  L1 spc L2         + 5 %
```

Run `20261025-043842-bls-a679c37e`: score 145.36 under `prog-cal` on `programmer-typed`, the best of three 160 M-evaluation searches started from the best 40 M result (`20261025-040445-bls-0aec6d68`, 145.75, same structure with a different right-hand consonant block).

- **Vowels on the left home row, consonants on the right**, as in the English optimum but mirrored. The four base punctuation keys sit together on the left bottom row: `' = , .` . = earns a base key; / and - do not.
- **One symbol layer.** Every symbol and digit off the base sits on one layer; the second layer holds only ~. Under the measured cost, a layer change is the expensive event, so everything goes where one hold reaches it. A run of symbols (`( ) ; , != , ->`) costs one press.
- **The layer's home row** holds `/ - ( : #` under the left hand and `_ ) * +` with two digits under the right. (and) sit on the two middle fingers, mirrored. `{ }` are adjacent on the left top row, `[ ]` on the right bottom row. `brackets` is 0 for every good design.
- **Digits are scattered** among the symbols (`9 8 6` top right, `3 2` on the right home row, `4 7 0 1` bottom left, 5 on a thumb), placed by frequency. Putting 1–0 in order on the layer's home row (`digits-row`, re-searched from the 40 M result: run `20261025-042045-bls-33240900`) costs **0.8 %**, about 0.05 % of typing time. That is a cheap price for a layout that can be learnt, so the digits-row variant is the one we recommend.

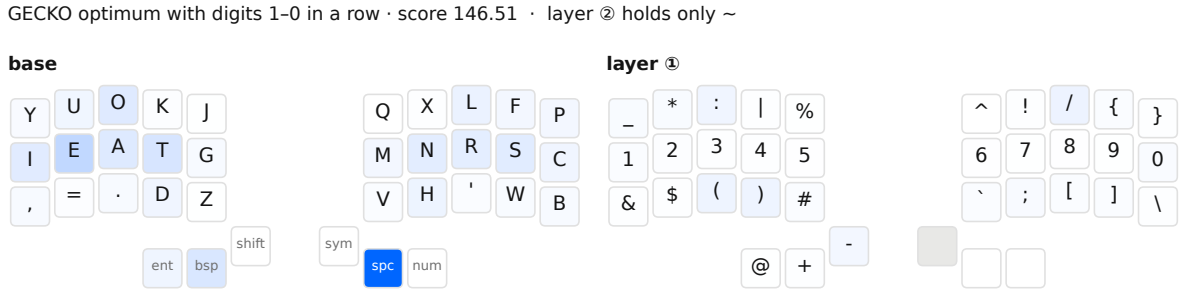


Figure 3: The recommended variant: digits 1–0 in order on the symbol layer's home row, symbols re-searched around them (score 146.51, +0.8 %).

4. Against published symbol layers

The three published designs share the optimum's thumbs and use Colemak-DH letters, so they differ only in the symbol layers. The cell `layer-cal` optimizes the layers and the free base keys with the same Colemak-DH letters pinned, which separates the layer design from the letters.

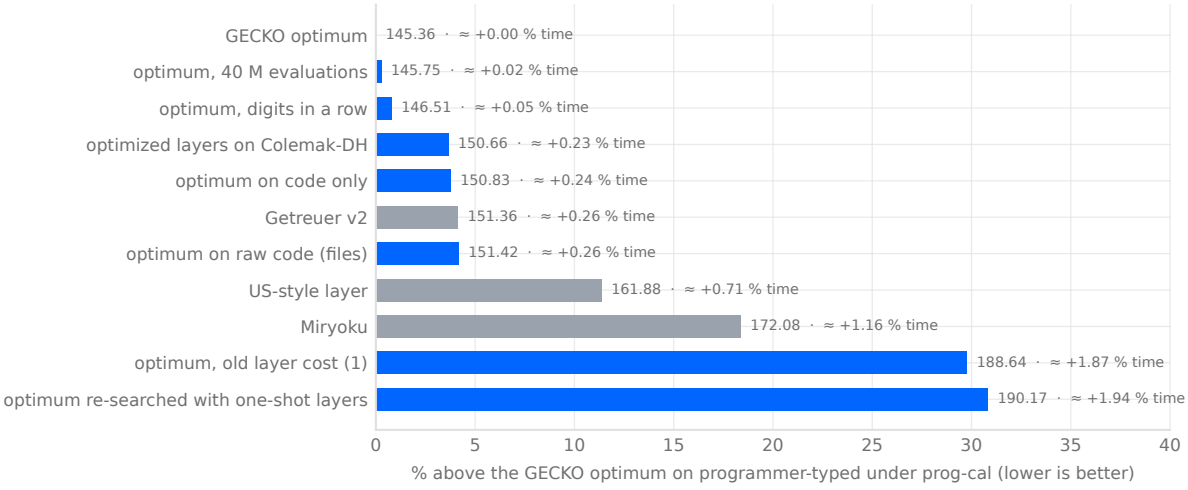


Figure 4: Scores on `programmer-typed` under `prog-cal`, as % above the optimum, with the gap in typing time per keystroke. Orange: the optimum; blue: GECKO variants; grey: published designs.

design	score	gap	≈ time	layers	sym. bigrams	brackets	SFB %
GECKO optimum	145.36	—	—	9.9	26.4	0	1.28
optimum, digits in a row	146.51	+0.8 %	+0.05 %	9.9	26.4	0	1.38
optimized layers on Colemak-DH	150.66	+3.6 %	+0.23 %	9.9	26.9	0	1.62
Getreuer v2	151.36	+4.1 %	+0.26 %	9.9	24.8	0	1.93

design	score	gap	≈ time	layers	sym. bigrams	brackets	SFB %
US-style layer	161.88	+11.4 %	+0.71 %	10.6	32.4	0	1.64
Miryoku	172.08	+18.4 %	+1.16 %	11.0	47.2	34	2.39
optimum, old layer cost	188.64	+29.8 %	+1.87 %	13.5	1.4	0	0.96

layers: layer presses per 100 bigrams; *sym. bigrams*: awkward symbol bigrams (`symbol_bigram`); *brackets*: bracket pairs neither adjacent nor mirrored.

- **Getreuer's hand-made layer is nearly optimal.** On the same letters, the optimized layer beats it by 0.5 % (0.03 % of time). It has the fewest awkward symbol bigrams of all, which is what its design rules aim at (inward rolls for `!=` `<=` `+=`). The SFB difference (1.93 vs 1.62) comes from its base: it keeps `,` `.` `'` where Colemak-DH has them, while the optimized layers move `=` and `'` onto base keys.
- **The letters matter more than the layer.** Re-optimizing the letters as well gains another 3.6 % (0.23 % of time), seven times what the optimized layer gains over Getreuer's.
- **Miryoku pays for its rules.** Symbols on one hand under the opposite thumb, in the shifted number-row order, give the most awkward symbol bigrams (47) and the only bracket cost (`{ }` and `[ ]` sit at the two ends of a row, neither adjacent nor mirrored). Its two layers (symbols and numbers) also need more presses: 11.0 per 100 bigrams. On the same letters it is 14 % behind the optimized layer.
- **The US-style layer** (the shifted number row on top) is 7 % behind the optimized layer on the same letters: its symbols follow digit order, not use.
- **All good designs press a layer about 9.9 times per 100 bigrams.** That floor is set by how often programmers type symbols that are not on the base; the designs differ in what happens *on* the layer.
- **One-shot layers lose.** Re-searching the 40 M optimum with one-shot layers (tap, then the key) ends at 190.17, +31 %: a one-shot costs a press for every symbol, a hold one per run. The calibration treats the tap like a held Shift, which may overcharge it, but runs of symbols are common in code (`()`, `;`, `->`, `!=`).

5. Which text?

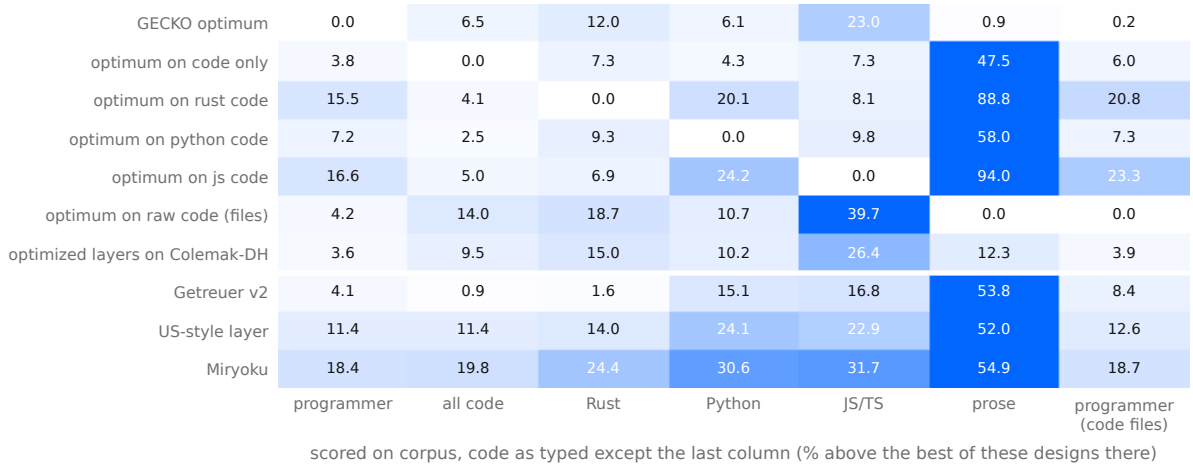


Figure 5: Each design (rows) scored on each corpus (columns), as % above the best of these designs there. Above the white line: GECKO optima, each searched on one text (the optimum on `programmer-typed`; the others on all typed code, one language, or code files); below: published designs. Code is as typed (auto-closed line closers dropped, identifiers completed after four characters) except in the last column.

- **Each language wants its own base symbols.** Optimized on one language's typed code (3 seeds each, the letters still on the base), the base takes that language's symbols: `:` `/` `=` `{` for Rust, `=` `_` `,` `.` for Python, `/` `{` `*` `=` for JS/TS. Each gains 6–23 % on its own language over the programmer optimum, and loses 7–17 % on the programmer mix and 58–94 % on prose.
- **Code alone is the wrong corpus.** Optimized on all typed code (`full-code`), the layout is 3.8 % behind on the programmer mix and 48 % behind on prose.
- **The published designs are code layouts.** With the same Colemak-DH letters as `layer-cal`, they are 52–55 % behind on prose where `layer-cal` is 12 %: they need 5.7 layer presses per 100 prose bigrams against 3.9, and have about twice the awkward symbol bigrams (Getreuer 51 against 28). Digits on a second layer are a likely cause (`10:30`, `(1)`, `$5` change layers twice). On code they do well: Getreuer is within 1.6 % of the Rust optimum and 0.9 % of the all-code optimum.
- **Typed or raw.** Optimizing on raw files (`full-raw`, the `programmer` corpus) is 4.2 % behind on typed text: the closing brace and long identifiers count for more in files than in typing.

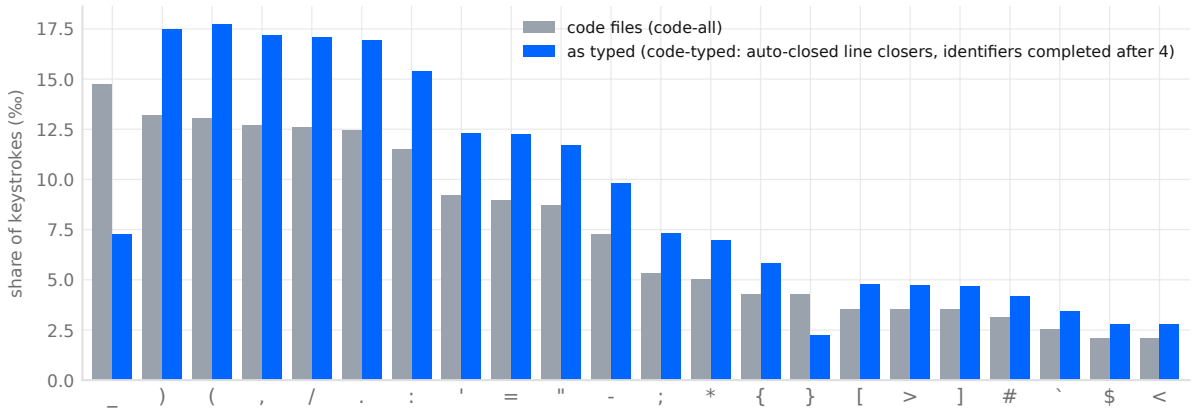


Figure 6: Symbol shares in code files and in code as typed. Most symbols gain share (typed code is 26 % shorter); } halves, because a closing brace that starts a line is inserted by the editor, and _ halves, because it sits inside long identifiers that are completed.

6. How sure is it?

Search. The layered problem (36 base keys and 72 layer keys for 26 letters, about 30 symbols, 10 digits and the thumb keys) is much harder for BLS than letters alone. At 40 M evaluations only 1 of 6 seeds reaches the best score of a cell, and seeds spread up to 3 %. With 4× the evaluations (3 seeds × 160 M from the best 40 M result) all three end within 0.25 % of each other, and the best is 0.27 % below the 40 M result (about 0.02 % of typing time). The structure (vowels left, one held symbol layer, digits among the symbols, ' = , . on the base) is the same in every one of these results; only the right-hand consonants and individual symbols move. So the reported optimum is near the best this cost function allows, but it is not proven optimal, and the exact placement of rare symbols is not meaningful.

Weights. The ranking of published designs follows the layer cost. Under the old weight, Miryoku's and the US-style layer's extra presses cost little, and all designs are within a few percent. The calibrated weight assumes a thumb-held layer costs what Shift costs; it is the decisive assumption of this report.

Timing. The score gaps between good designs are small in time: the optimum, Getreuer and the optimized layer on Colemak-DH are within 0.26 % of typing time per keystroke of each other, below what any typing study could detect. The gaps to the US-style layer (0.7 %), Miryoku (1.2 %) and a layer with base punctuation moved (1.9 %) are larger but still small.

7. Conclusions

1. **Measure the layer cost.** A held Shift costs ×1.78 the interval, 3.7 SFBs. With the old weight an optimizer puts , . ' on a layer; with the measured one it keeps them on the base. Any symbol-layer optimizer needs this number.
2. **One symbol layer, held, reached by one thumb.** Two layers or one-shot keys cost presses. Digits in a row on it cost 0.8 %.
3. **Published designs are close to optimal when they follow use.** Getreuer's layer is within 0.5 % of an optimized layer on the same letters. Miryoku's symmetric rules cost 14 %, a US-style layer 7 %.
4. **Letters matter more than the layer**, and code alone is the wrong corpus for a programmer: optimize on typed code mixed with prose. A single-language optimum gains 6–23 % on its language and loses 7–17 % on the mix.

Limits. One board (corne-36); one cost function, whose layer weight comes from Shift, not from a thumb-held layer; synthetic editor typing (no IDE-specific completion, no snippets); and the search is not exact (seeds at 4× the budget agree within 0.25 %). No speed claim is made: differences between good designs are below 0.3 % of typing time.

8. Reproduce

```
cd bench/programming-layout-2026-10-25
python3 -I refs.py            # reference designs → layouts/
./grid.sh && ./grid2.sh       # 6 seeds × BLS 40 M per cell → seeds-*.json
python3 -I analyze.py         # → results.json
MPLCONFIGDIR=/tmp/mpl python3 -I ../../docs/papers/programming-layout/figures.py
python3 -I ../../docs/papers/programming-layout/build.py
gecko verify run:20261025-043842-bls-a679c37e
```